

MVP: Multiple View Prediction Improves GUI Grounding

Yunzhu Zhang¹ Zeyu Pan² Zhengwen Zeng³ Shuheng Shen^{3*} Changhua Meng³ Linchao Zhu^{1*}

¹College of Computer Science and Technology, Zhejiang University

²College of Computer Science and Technology, Hangzhou Dianzi University

³Venus Team, Ant Group

{yunzhuzhang0918, zhulinchao7}@gmail.com, panzeyucs@hdu.edu.cn,

{zengzhengwen.zzw, shuheng.ssh, changhua.mch}@antgroup.com

Abstract

GUI grounding, which translates natural language instructions into precise pixel coordinates, is essential for developing practical GUI agents. However, we observe that existing grounding models exhibit significant **coordinate prediction instability**—minor visual perturbations (e.g., cropping a few pixels) can drastically alter predictions, flipping results between correct and incorrect. This instability severely undermines model performance, especially for samples with high-resolution and small UI elements. To address this issue, we propose **Multi-View Prediction (MVP)**, a training-free framework that enhances grounding performance through multi-view inference. Our key insight is that while single-view predictions may be unstable, aggregating predictions from multiple carefully cropped views can effectively distinguish correct coordinates from outliers. MVP comprises two components: (1) **Attention-Guided View Proposal**, which derives diverse views guided by instruction-to-image attention scores, and (2) **Multi-Coordinates Clustering**, which ensembles predictions by selecting the centroid of the densest spatial cluster. Extensive experiments demonstrate MVP’s effectiveness across various models and benchmarks. Notably, on ScreenSpot-Pro, MVP boosts UI-TARS-1.5-7B to 56.1%, GTA1-7B to 61.7%, Qwen3VL-8B-Instruct to 65.3%, and Qwen3VL-32B-Instruct to 74.0%. The code is available at <https://github.com/ZJUSCL/MVP>.

1. Introduction

The development of automated agents for graphical user interfaces (GUIs) represents a pivotal frontier in artificial general intelligence (AGI) research [8, 21, 28, 30, 31, 45, 51]. These agents fundamentally rely on GUI grounding, which mappings natural language instructions to their correspond-

ing actionable elements within screenshots or live interfaces [12, 14, 17, 33].

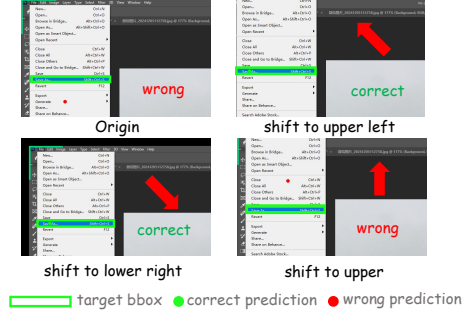
GUI grounding models are built upon Large Vision Language Models (LVLMs), typically formulating GUI grounding as a generation task, where models output pixel coordinates as text tokens (e.g., “x=123, y=456”) [6, 10, 24]. However, it is inherently challenging for language models to establish a robust correspondence between visual elements and text coordinates tokens based on instructions [32, 34, 50]. Despite extensive training on GUI images through supervised fine-tuning (SFT) or reinforcement learning (RL), grounding models still generate unexplainable erroneous coordinates, particularly when facing high-resolution images and small target UI elements that are difficult to identify.

We carefully analyze the failure cases and discover that an incorrect prediction does not mean the model lacks the capability to locate the target. Rather, the models suffer from **prediction instability**, where minimal perturbations to input images (e.g., shifting by a few pixels) cause dramatic changes in predicted coordinates. As shown in Figure 1(a), such minor visual variations can flip predictions between correct and incorrect states, revealing high sensitivity to input perturbations.

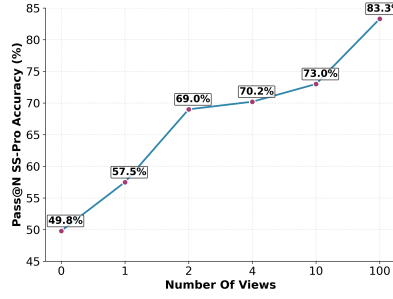
This observation suggests that single full-screenshot inference inadequately unleashes the model’s true grounding capability. To verify this hypothesis, we conduct a preliminary experiment. Specifically, we randomly crop multiple 1280×720 sub-regions from the original ScreenSpot-Pro [14] screenshots, ensuring each view contains the target bounding box. We then predict coordinates for each view. As shown in Figure 1(b), the pass@N accuracy (whether at least one prediction among N views is correct) consistently improves as the number of views increases. This motivate us to leverage multiple sub-regions during inference to improve prediction performance.

Based on this observation, we propose the Multiple View Prediction (MVP) framework. It operates in two

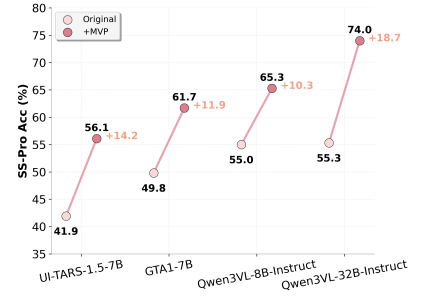
*Corresponding Author.



(a) Example of prediction instability.



(b) Pass@N increases with number of views.



(c) MVP significantly boosts performance.

Figure 1. (a) An example of model’s prediction instability from ScreenSpot-Pro. The instruction is “save image in a specific format”. Slightly shifting the screenshot causes significantly different predicted coordinates. (b) We crop different views from the original screenshots in ScreenSpot-Pro and then perform inference separately on them using GTA1-7B. The pass@N accuracy improves with number of views increasing, indicating the model possessing the ability to predict the correct prediction. (c) Our MVP significantly improves performance of different architectures and sizes grounding models by aggregating results of different views.

key stages: Attention-Guided View Proposal and Multi-Coordinate Clustering. First, MVP generates multiple views by cropping sub-regions from the original screenshot, using instruction-to-image attention scores to guide the process. These views maintain diversity while ensuring a high likelihood of containing the target UI elements. Each resulting view, along with the original image, undergoes independent inference to yield multiple coordinate predictions. Finally, the Multi-Coordinate Clustering component aggregates these results by performing spatial clustering on all predicted coordinates and outputs the centroid of the largest cluster as the final prediction.

The core intuition behind MVP is to mitigate prediction instability through multi-view integration. Although individual view predictions may be unreliable, they usually exhibit spatial patterns that the incorrect coordinates tend to scatter arbitrarily whereas the correct ones consistently fall within the target bounding box region. By clustering predictions from diverse views and identifying the densest cluster, MVP effectively distinguishes reliable coordinates from outliers, thereby enhancing grounding performance.

MVP is a training-free framework that can be easily integrated with different grounding models, such as GTA1-7B [41], UI-TARS-1.5-7B [26], and Qwen3VL-{8B, 32B}-Instruct [25], spanning from 7B/8B to 32B parameter scales. Experimental results on ScreenSpot-Pro [14], UI-Vision [20] and OS-World-G [37] benchmarks demonstrate that MVP can significantly improve existing grounding models’ performance.

Contributions. Our contributions are threefold:

- We identify coordinate prediction instability in grounding models, which severely undermines model performance.
- We propose Multi-View Prediction (MVP), a training-free framework that aggregates predictions from multiple

attention-guided views through spatial clustering to mitigate prediction instability.

- We demonstrate that MVP can integrate with grounding models of different architectures, improving accuracy on three challenging grounding benchmarks.

2. Related Work

GUI grounding, the task of mapping natural language instructions to precise coordinates, is a core capability for developing GUI agents capable of real-world application.

The task was first introduced by SeeClick [6] through the ScreenSpot benchmark, showing that grounding pretraining improves end-to-end success across mobile, web, and desktop UIs scenarios. Subsequent research in this area can be broadly categorized into following directions:

- **Direct Coordinate Optimization** methods enhance the model’s grounding ability by supervised fine-tuning (SFT) on large-scale GUI-specific datasets [7, 16, 24, 35, 42], or employing reinforcement learning (RL) with rule-based rewards [18, 27, 41], directly optimizing the output coordinates tokens. While these methods improve performance, they require substantial computational resources for training and still struggle with high resolution and small UI elements. Unlike these resource-intensive approaches, MVP requires no additional training.
- **Iterative Zoom-in** methods reframe grounding as a multi-step decision process. These approaches either leverage execution feedback from GUI agents [19] or exploit the model’s own reasoning capability [5, 13, 43, 50] to iteratively narrow down to a correct sub-region and then make the final prediction. However, these methods suffer from (1) error accumulation, where a mistake in an early step propagates to subsequent stages; (2) additional training or requiring feedback from external agents. In

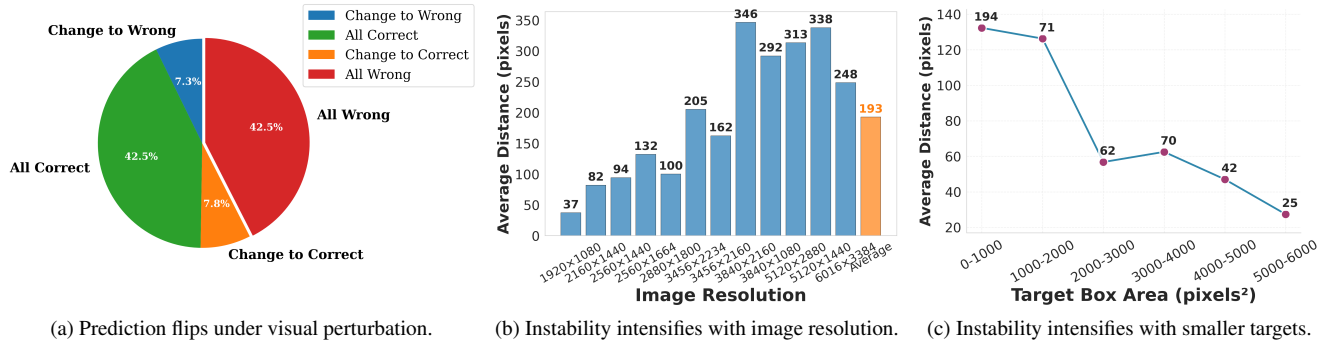


Figure 2. We evaluate model instability by adding a 28-pixel border to ScreenSpot-Pro images and performing separate inference runs with GTA1-7B. (a) This minor visual perturbation causes 7.3% of originally correct predictions to become incorrect, and 7.8% of originally wrong predictions to become correct, revealing high sensitivity to input variations. (b) When analyzing the distance between the two predicted coordinates grouped by image resolution, we observe that instability increases significantly with higher resolutions. (c) Similarly, when grouping by the area of the target region, we find that instability is more pronounced for smaller UI elements.

contrast to this sequential search for one optimal view, our MVP employs a parallel multi-view strategy, aggregating predictions via clustering, thereby avoiding error propagation. Meanwhile, MVP operates in a fully training-free manner without relying on any external feedback.

- **Attention-Based** methods leverage the intrinsic instruction-spatial alignment in LVLMS. They derive cross-attention scores from transformer layers to identify the most relevant visual patch and directly output its center coordinates [4, 34, 39]. However, these methods are highly dependent on the precision of the attention scores, limiting their generalizability across different instructions. Different from these methods, MVP preserves the standard text generation paradigm as well as better generalization across diverse scenarios.

3. Preliminary Analysis

In this section, we systematically diagnose the prediction instability in GUI grounding models. We experiment with GTA1-7B model [41] on the ScreenSpot-Pro benchmark [14] to demonstrate that this instability severely limits model reliability and then discuss on its underlying causes.

3.1. Single Inference is Unreliable

Our core finding is that grounding models are highly sensitive to visual perturbations, making single-inference results unreliable. Specifically, by adding a mere 28-pixel border (significantly smaller than the image resolution) to screenshots, we observe drastically different coordinate predictions from the same model: the average coordinate shift of 193 pixels far exceeds the size of typical UI elements in ScreenSpot-Pro (Figure 2(b)).

Crucially, this instability directly impacts accuracy. As shown in Figure 2(a), the model achieved 57.5% accuracy

when considering at least one of the two predictions as correct—significantly higher than its 49.8% single-prediction accuracy. This gap confirms that the model possesses the requisite capability, but single-view inference fails to harness it consistently.

3.2. What Drives Instability

To understand what makes predictions unstable, we analyze how instability varies with input resolution and target UI element size. Figure 2(b) and Figure 2(c) reveal a clear trend: instability intensifies dramatically for (1) high-resolution screenshots and (2) samples with small target elements.

We attribute these challenges to both architectural and data-driven limitations. At the architectural level, two model-side factors contribute to the observed instability patterns. First, RoPE’s extrapolation limitations may degrade performance on high-resolution images, where position indices exceed the training distribution, causing minor spatial changes to yield vastly different token sequences. Second, the projector that transforms visual features into visual tokens may discard fine-grained spatial information, limiting the model’s ability to perceive small targets accurately. Beyond these architectural factors, current training datasets lack sufficient examples of high-resolution screenshots and small UI element samples, creating a generalization short-fall at test time.

This diagnosis naturally leads to our solution: if a single view is unreliable, but the model can sometimes predict correctly, then aggregating predictions from multiple views should yield more robust and accurate results.

4. Methods

In this section, we propose the training-free Multiple View Prediction (MVP) framework to address the identified in-

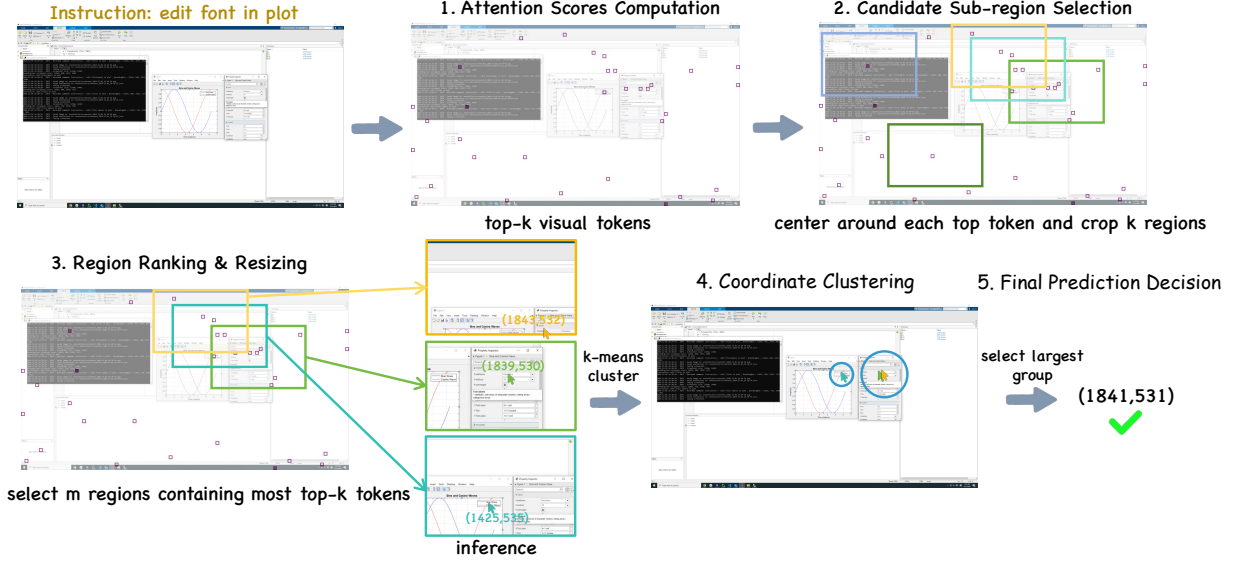


Figure 3. Overview of our Multiple View Prediction pipeline, which consists of two main stages: Attention-Guided View Proposal and Multi-Coordinate Clustering. First, MVP takes the user instruction and screenshot, forwarding through the language model to derive attention scores from the instruction to each visual token. Then the top-k scores tokens are selected, and an $h \times w$ sub-region is cropped around the center of each corresponding visual patch. These sub-regions are ranked by the number of top-k tokens they contain. The top-m regions are chosen and enlarged to form the final set of views. The model independently predicts coordinates for each view. Finally, MVP aggregates all the predictions by clustering the coordinates based on spatial proximity and outputs the center of the largest cluster as the final prediction.

stability issues. The MVP framework consists of two main components: (1) Attention-Guided View Proposal identifies sub-regions containing the target bounding box and generates different views, reducing input resolution and enhancing small target visibility (2) Multi-Coordinate Clustering aggregates the predictions from multiple views and determines the final coordinates, identifying the spatially consistent cluster to filter out outliers and enhance robustness.

4.1. Attention-Guided View Proposal

This module aims to generate multiple views by locating sub-regions that contain the target UI elements, guided by cross-attention scores. It leverages the strong text-visual alignment capability of LVLMs, whose attention mechanisms in middle-to-deep layers can effectively localize instruction-relevant regions [11, 38, 39, 48]. Given the system prompt, a GUI screenshot, and the user instruction, this module outputs m cropped image regions. The process consists of three main steps: Attention Score Computation, Candidate Sub-region Selection, and Region Ranking & Resizing.

Attention Score Computation. We compute the cross-attention scores using the text token as the query and the visual tokens as the keys. Specifically, we use the center comma token (“,”) from the predicted coordinate format (e.g. “(123,456)”) as the query token, as it demonstrates

better region localization performance, with further analysis provided in the Appendix. Let $V \in \mathbb{R}^{H \times L_v \times d}$ represent the visual tokens and $T_{\text{comma}} \in \mathbb{R}^{H \times 1 \times d}$ denote the comma token. The cross-attention scores are computed as:

$$A = \text{Softmax} \left(\frac{T_{\text{comma}} V^T}{\sqrt{d}} \right), A \in \mathbb{R}^{H \times L_v} \quad (1)$$

The final attention score assigned to each visual token is obtained by averaging across all attention heads:

$$\text{scores} = \frac{1}{H} \sum_{i=1}^H A[i, :], \text{scores} \in \mathbb{R}^{L_v} \quad (2)$$

where L_v is the number of visual tokens, H is the number of attention heads, and d is the dimension of the model.

Candidate Sub-region Selection. We select k candidate regions based on the computed attention scores. Specifically, we choose the top- k visual tokens with the highest scores. Let $\mathcal{T} = \{(t_j, x_j, y_j) \mid j = 1, \dots, k\}$ denote the set of top- k visual tokens and their corresponding patch center coordinates, for each top- k token, we crop an $h \times w$ sub-region centered at (x_i, y_i) in the original image, resulting in k candidate regions:

$$R_i = \text{Crop} \left(I, x_i - \frac{w}{2}, y_i - \frac{h}{2}, w, h \right), i \in [1, k] \quad (3)$$

Algorithm 1 Attention-Guided View Proposal

Require: Text instruction T , original image I , view size (h, w) , view number m , resize ratio α

Ensure: Candidate views set $\mathcal{V} = \{R_1^{\text{resized}}, \dots, R_m^{\text{resized}}\}$

- 1: **1. Attention Score Computation**
 - 2: Extract visual tokens $V \in \mathbb{R}^{H \times L_v \times d}$ from I
 - 3: Get comma token $T_{\text{comma}} \in \mathbb{R}^{H \times 1 \times d}$ as query token
 - 4: Use Eq 1. and Eq 2. to compute attention scores
 - 5: **2. Candidate Sub-region Selection**
 - 6: Sort visual tokens by scores in descending order
 - 7: Select top- k tokens: $\text{tokens} = \{t_1, t_2, \dots, t_k\}$
 - 8: Get corresponding positions: $\{(x_1, y_1), \dots, (x_k, y_k)\}$
 - 9: Initialize empty region set $\mathcal{R} = \emptyset$
 - 10: **for** each selected token t_i at position (x_i, y_i) **do**
 - 11: Crop candidate region $\{R_i\}$ using Eq 3.
 - 12: Compute ranking score $\text{rank}(R_i)$ using Eq 4.
 - 13: $\mathcal{R} = \mathcal{R} \cup \{R_i\}$
 - 14: **end for**
 - 15: **3. Region Ranking & Resizing**
 - 16: Sort regions by ranking scores: $R_{(1)}, R_{(2)}, \dots, R_{(k)}$
 - 17: Select top- m regions: $\mathcal{V} = \{R_{(1)}, \dots, R_{(m)}\}$
 - 18: Resize each region $R_{(i)}$ in selected set using Eq 5.
 - 19: **return** $\mathcal{V}$
-

where I is the original GUI image and $\text{Crop}(I, x, y, w, h)$ function extracts a rectangular region.

Region Ranking & Resizing. We select m regions from k candidates to form the final views. Regions containing more top- k visual tokens are considered more likely to contain the target bounding box. We rank the candidate regions by the number of these tokens whose patch center coordinates fall within the region, and choose the top- m regions:

$$\text{rank}(R_i) = \sum_{j=1}^k \mathbb{I}[(x_j, y_j) \in R_i], i \in [1, k] \quad (4)$$

Considering small UI elements poses more instability to grounding models (Section 3.2), we enlarge the selected regions to enhance the visibility of small targets:

$$R_i^{\text{resized}} = \text{Resize}(R_i, \alpha h, \alpha w), \quad \alpha > 1 \quad (5)$$

4.2. Multi-Coordinate Clustering

This module takes the m cropped views as input and outputs the final predicted coordinate. It firstly performs inference on each of the m views along with the original full image, yielding $m + 1$ coordinate predictions $(x_i, y_i)_{i=1}^{m+1}$, then identifies the correct prediction by clustering spatially consistent coordinates and filtering out outliers. This process consists of two steps: Coordinate Clustering and Final Prediction Decision.

Algorithm 2 Multi-Coordinate Clustering

Require: Distance threshold τ , coordinate set $\mathcal{C} = \{p_1, p_2, \dots, p_{m+1}\}$

Ensure: Final coordinate p_{final}

- 1: **Step 1: Cluster coordinates**
 - 2: Initialize clusters $\mathcal{G} = \{\}$, unassigned $U = \mathcal{C}$
 - 3: **while** $U \neq \emptyset$ **do**
 - 4: $p_{\text{seed}} \leftarrow U[0], G \leftarrow \{p_{\text{seed}}\}, U \leftarrow U \setminus \{p_{\text{seed}}\}$
 - 5: **repeat**
 - 6: $G_{\text{prev}} \leftarrow G$
 - 7: **for** $p \in U$ **do**
 - 8: $\text{center} \leftarrow \frac{1}{|G|} \sum_{q \in G} q$
 - 9: **if** $\|p - \text{center}\|_2 \leq \tau$ **then**
 - 10: $G \leftarrow G \cup \{p\}, U \leftarrow U \setminus \{p\}$
 - 11: **end if**
 - 12: **end for**
 - 13: **until** $G = G_{\text{prev}}$
 - 14: $\mathcal{G} \leftarrow \mathcal{G} \cup \{G\}$
 - 15: **end while**
 - 16: **Step 2: Final Prediction Decision**
 - 17: $G^* = \arg \max_{G \in \mathcal{G}} |G|$
 - 18: **if** $\exists$ multiple G with max size **then**
 - 19: $G^* = \arg \max_{G_k \in \mathcal{G}} \sum \text{rank}(R_i), p_i \in G_k$
 - 20: **end if**
 - 21: $p_{\text{final}} = \frac{1}{|G^*|} \sum_{p_i \in G^*} p_i$
 - 22: **return** p_{final}
-

Coordinate Clustering We cluster the coordinate predictions using K-means based on the distance. The metric between any two predictions p_i and p_j is calculated as:

$$d(p_i, p_j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} \quad (6)$$

Final Prediction Decision The reliability of a prediction cluster G_k is determined by its size $|G_k|$ —while incorrect predictions may scatter arbitrarily, correct ones are spatially consistent as they all fall within the target bounding box. We select the center coordinates of the largest cluster as the final prediction:

$$G^* = \arg \max_{G_k} |G_k|, \quad (x_{\text{final}}, y_{\text{final}}) = \frac{1}{|G^*|} \sum_{p_i \in G^*} p_i \quad (7)$$

In cases when multiple clusters have the same maximum size, we leverage the attention-based ranking for decision, selecting the cluster whose points corresponding regions containing the most top- k visual tokens:

$$G^* = \arg \max_{G_k \in \mathcal{G}} \sum \text{rank}(R_i), p_i \in G_k \quad (8)$$

Model	Development	Creative	CAD	Scientific	Office	OS	Overall
Closed-source Models							
GPT-4o [22]	0.7	0.6	1.5	1.2	0.9	0.0	0.8
Claude Computer Use [1]	12.6	16.8	11.9	25.8	26.9	8.1	17.1
UI-TARS-1.5 [26]	63.9	50.4	58.2	69.3	79.6	51.0	61.6
Seed1.5-VL [9]	53.8	59.2	59.0	61.4	74.8	60.2	60.9
Open-Source Models							
SeeClick-7B [6]	0.3	0.6	1.9	2.0	0.9	1.5	1.1
UGround-V1-7B [7]	28.1	31.7	14.6	39.0	49.6	24.5	31.1
UGround-V1-72B [7]	31.1	35.8	13.8	50.0	51.3	25.5	34.5
Qwen2.5-VL-32B-Instruct [3]	48.8	42.2	31.0	55.5	64.3	50.5	48.0
RegionFocus (Qwen2.5VL-72B) [19]	51.2	57.2	60.9	66.5	80.9	57.1	61.6
GTA1-72B [41]	57.2	51.0	49.8	63.0	77.0	57.1	58.4
GUI-Actor-2.5VL-7B [34]	38.1	41.3	38.3	50.8	63.0	38.8	44.6
SE-GUI-7B [44]	44.5	37.2	42.1	54.7	70.4	38.8	47.2
UI-Venus-72B [8]	59.5	55.4	57.5	66.5	77.8	57.7	61.9
V2P-7B [4]	46.8	43.1	47.1	56.3	68.3	45.4	50.6
GMS (Gemini-2.5-Flash-Lite) [15]	44.8	54.8	57.5	55.9	70.4	44.9	54.6
GUI-Spotlight [13]	53.3	44.4	51.0	52.4	71.3	46.9	52.8
GUI-Cursor-7B [49]	57.5	45.8	53.2	61.4	74.8	50.0	56.5
UI-INS-32B [5]	55.8	46.4	48.4	62.2	80.0	54.1	57.0
HyperClick [47]	46.9	45.1	48.5	56.7	60.9	40.8	48.2
UI-TARS-1.5-7B	36.4	38.1	20.5	49.6	68.7	31.5	41.9
+ MVP	51.8 $\uparrow$ 15.4	50.0 $\uparrow$ 11.9	53.3 $\uparrow$ 32.8	57.9 $\uparrow$ 8.3	73.0 $\uparrow$ 4.3	54.6 $\uparrow$ 23.1	56.1 $\uparrow$ 14.2
GTA1-7B	43.4	44.8	44.4	55.9	74.8	35.2	49.8
+ MVP	58.9 $\uparrow$ 15.5	52.6 $\uparrow$ 7.8	60.2 $\uparrow$ 15.8	63.0 $\uparrow$ 7.1	79.1 $\uparrow$ 4.3	56.1 $\uparrow$ 20.9	61.7 $\uparrow$ 11.9
Qwen3VL-8B-Instruct	52.8	49.1	49.0	56.7	75.2	50.5	55.0
+ MVP	61.5 $\uparrow$ 8.7	60.2 $\uparrow$ 11.1	61.3 $\uparrow$ 12.3	67.3 $\uparrow$ 10.6	82.6 $\uparrow$ 7.4	62.8 $\uparrow$ 12.3	65.3 $\uparrow$ 10.3
Qwen3VL-32B-Instruct	43.1	54.4	57.5	62.6	73.0	42.3	55.3
+ MVP	71.6 $\uparrow$ 28.5	69.3 $\uparrow$ 14.9	74.7 $\uparrow$ 17.2	70.5 $\uparrow$ 7.9	87.4 $\uparrow$ 14.4	73.5 $\uparrow$ 31.2	74.0 $\uparrow$ 18.7

Table 1. Evaluation results on the ScreenSpot-Pro benchmark. Baseline models are evaluated using official instructions, while other models’ results are sourced from the benchmark leaderboard. Our method shows significant performance improvements across all categories. These consistent gains across diverse model architectures validate MVP’s effectiveness in addressing high-resolution GUI grounding challenges through its multi-view prediction mechanism.

5. Experiments

5.1. Models and Benchmarks

We conduct comprehensive evaluations across diverse grounding models and scales to assess the generalizability and effectiveness of our MVP framework, including UI-TARS-1.5-7B [26], GTA1-7B [41] and Qwen3VL-{8B, 32B}-Instruct [25]. The models are evaluated on three challenging GUI grounding benchmarks: (1) ScreenSpot-Pro [14], a benchmark for evaluating visual grounding on high-resolution screenshots of professional software, covering application domains such as creative tools, office. (2) UI-Vision [20], a diverse benchmark which samples from 83 applications across 6 domains with dense referring expressions platforms. (3) OS-World-G [37], a real-world interactive benchmark containing 564 screenshots from OS-World [36], focusing on operating-system tasks like file operations and app launching.

5.2. Implementation Details

We run 7B/8B model on 8 RTX4090 and 32B model on 4 A100. The view size (h, w) is set to 1280×720 for

all experiments, and then are resized to 2560×1440 . The view number m is set to 4 for GTA1-7B and UI-TARS-1.5-7B, and 2 for Qwen3VL-{8B,32B}-Instruct by default. For screenshots with resolution lower than 720P, we generate different views by adding 28-pixel black border to the left, right, upper and bottom of the image accordingly, or we apply the Attention Heuristic Cropping to crop views. According to FlexSelect [48], we choose attention scores from layer 20 of the language model for GTA1-7B and UI-TARS-1.5-7B, layer 24 for Qwen3VL-8B-Instruct and layer 48 for Qwen3VL-32B-Instruct to crop views. Top-100 visual tokens are selected to crop candidate regions. The K-means threshold τ is set to 14 pixels for all models. The evaluation prompt follows the model’s official GitHub repository.

5.3. Main Results

Results on ScreenSpot-Pro Grounding models exhibit greater instability when processing high-resolution screenshots, leading to suboptimal performance. Our MVP framework effectively addresses this issue through its multi-view prediction mechanism, as demonstrated by consistent performance gains across all categories and model scales on

Model	Text Match	Element Recognition	Layout Understanding	Fine-grained Manipulation	Refusal	Average
Closed-source Models						
Operator [23]	51.3	42.4	46.6	31.5	-	40.6
Gemini-2.5-pro [29]	59.8	45.5	49.0	33.6	38.9	45.2
Seed1.5-VL [9]	73.9	66.7	69.6	47.0	18.5	62.9
Open-Source Models						
Qwen2.5-VL-72B [3]	52.6	74.6	74.7	55.3	-	62.2
UI-TARS-72B [24]	69.4	60.6	62.9	45.6	-	57.1
UI-Venus-Ground-72B [8]	82.1	71.2	70.7	64.4	-	70.4
OS-Atlas-7B [35]	44.1	29.4	35.2	16.8	7.4	27.7
UGround-V1-7B [7]	51.3	40.3	43.5	24.8	-	36.4
Aguvis-7B [40]	55.9	41.2	43.9	28.2	-	38.7
Jedi-7B [37]	65.9	55.5	57.7	46.9	7.4	54.1
GTA1-72B [41]	57.9	76.9	77.3	66.7	-	66.7
GUI-Spotlight [13]	68.2	60.6	63.2	45.6	-	62.7
UI-TARS-1.5-7B	67.3	64.5	65.2	42.9	-	61.9
+ MVP	78.5 $\uparrow$ 11.2	69.1 $\uparrow$ 4.6	70.0 $\uparrow$ 4.8	59.7 $\uparrow$ 16.8	-	66.8 $\uparrow$ 4.9
GTA1-7B	80.4	69.4	69.1	60.4	-	67.5
+ MVP	80.8 $\uparrow$ 0.4	71.6 $\uparrow$ 2.2	71.1 $\uparrow$ 2.0	61.0 $\uparrow$ 0.6	-	68.7 $\uparrow$ 1.2
Qwen3VL-8B-Instruct	79.7	71.8	74.3	59.1	-	68.8
+ MVP	82.4 $\uparrow$ 2.7	76.7 $\uparrow$ 4.9	79.5 $\uparrow$ 5.2	61.7 $\uparrow$ 2.6	-	72.7 $\uparrow$ 3.9
Qwen3VL-32B-Instruct	81.6	75.7	77.1	62.4	-	71.7
+ MVP	81.6	76.4 $\uparrow$ 0.7	77.5 $\uparrow$ 0.4	62.4	-	72.0 $\uparrow$ 0.3

Table 2. Evaluation results on the OS-World-G benchmark. The baseline results are evaluated using the corresponding official GitHub instructions, while other models’ results are sourced directly from the UI-Venus Technical Report [8]. These experimental results demonstrate the effectiveness of our MVP framework in real-world interactive scenarios.

ScreenSpot-Pro (Table 1). Specifically, MVP improves GTA1-7B by 11.9 points (49.8 $\rightarrow$ 61.7), UI-TARS-1.5-7B by 14.2 points (41.9 $\rightarrow$ 56.1), and Qwen3VL-8B-Instruct by 10.3 points (55.0 $\rightarrow$ 65.3). Most notably, when integrated with Qwen3VL-32B-Instruct, MVP elevates performance from 55.3 to 74.0, establishing new state-of-the-art results that surpass all existing open-source and closed-source models. These substantial improvements across diverse architectures confirm MVP’s effectiveness in addressing the challenges of high-resolution GUI grounding through its multi-view prediction mechanism.

Results on OS-World-G As shown in Table 2, our MVP framework demonstrates consistent performance improvements on OS-World-G, enhancing the average scores of UI-TARS-1.5-7B by 4.9 points (61.9 $\rightarrow$ 66.8), GTA1-7B by 1.2 points (67.5 $\rightarrow$ 68.7), Qwen3VL-8B-Instruct by 3.9 points (68.8 $\rightarrow$ 72.7), and Qwen3VL-32B-Instruct by 0.3 points (71.7 $\rightarrow$ 72.0). These results validate MVP’s effectiveness in real-world interactive scenarios. The smaller performance gains on OS-World-G relative to UI-Vision and ScreenSpot-Pro stem from its lower-resolution (720P/1080P), which inherently poses less instability to models and consequently limits improvement potential.

Results on UI-Vision As shown in Table 3, our MVP framework consistently enhances performance across all UI-Vision categories. For 7B models, MVP brings aver-

age gains of +3.4 (UI-TARS-1.5-7B) and +4.0 (GTA1-7B) points, with Qwen3VL-8B-Instruct achieving +4.7 points improvement. Most notably, Qwen3VL-32B-Instruct with MVP establishes new state-of-the-art performance with +7.7 points overall (36.4 $\rightarrow$ 44.1), surpassing even the 72B UI-Venus-Ground-72B (36.8) by +7.3 points. These results demonstrate that our method effectively improves model performance across different applications and platforms, showing strong generalizability across various domains.

5.4. Ablations

Influence of Attention-Guided View Proposal This study ablates the Attention-Guided View Proposal module, which generates diverse views by preserving semantically rich regions at a reduced input resolution. We compare it against a Border Padding strategy that creates views by adding 28-pixel borders around the original screenshot. Results in Table 4 show that while Border Padding improves upon the baseline, our Attention-Guided View Proposal method delivers a significantly larger gain. This demonstrates that our attention-guided strategy for selecting informative sub-regions is effective on view generation.

Influence of Multi-Coordinate Clustering This ablation study evaluates the efficacy of our Multi-Coordinate Clustering module in mitigating prediction instability by aggregating coordinates from multiple views. We compare it against two alternative strategies: (1) randomly selecting one prediction, and (2) averaging over all predicted coordi-

Model	Basic	Functional	Spatial	Average
Closed-source Models				
GPT-4o [22]	1.6	1.5	1.0	1.4
Claude-3.7-Sonnet [2]	9.5	7.7	7.6	8.3
Open-Source Models				
Qwen2.5-VL-7B [3]	1.2	0.8	0.5	0.9
UI-TARS-7B [24]	20.1	24.3	8.4	17.6
UI-TARS-72B [24]	31.4	30.5	14.7	25.5
UI-Venus-Ground-7B [8]	36.1	32.8	11.9	26.5
UI-Venus-Ground-72B [8]	45.6	42.3	23.7	36.8
OS-Atlas-7B [35]	12.2	11.2	3.7	9.0
Phi-Ground-7B-16C-DPO [46]	36.8	37.1	7.6	27.2
UGround-V1-7B [7]	15.4	17.1	6.3	12.9
HyperClick [47]	35.3	32.1	11.0	25.7
GUI-Spotlight [13]	32.1	30.2	9.1	23.4
UI-TARS-1.5-7B	29.3	27.4	10.0	22.2
+ MVP	31.8 $\uparrow$ 2.5	31.8 $\uparrow$ 4.4	13.2 $\uparrow$ 3.2	25.6 $\uparrow$ 3.4
GTA1-7B	35.2	32.9	11.5	26.5
+ MVP	38.9 $\uparrow$ 3.7	37.7 $\uparrow$ 4.8	14.9 $\uparrow$ 3.4	30.5 $\uparrow$ 4.0
Qwen3VL-8B-Instruct	32.9	34.1	14.7	27.2
+ MVP	38.3 $\uparrow$ 5.4	38.8 $\uparrow$ 4.7	18.7 $\uparrow$ 4.0	31.9 $\uparrow$ 4.7
Qwen3VL-32B-Instruct	43.9	42.8	22.4	36.4
+ MVP	49.4 $\uparrow$ 5.5	52.0 $\uparrow$ 9.2	30.8 $\uparrow$ 8.4	44.1 $\uparrow$ 7.7

Table 3. Evaluation results on UI-Vision. Baseline models are evaluated using the official GitHub instructions, while results for other models are taken directly from the UI-Vision leaderboard [20]. Our method effectively improves model performance across diverse applications, exhibiting strong generalizability.

View Proposal Method	Number of Views	SS-Pro Avg.
Baseline (Single Full Image)	–	49.8
Border Padding	4	57.3
Attention-Guided View Proposal	4	61.7

Table 4. Comparison of view generation methods on the ScreenSpot-Pro benchmark using GTA1-7B. Our Attention Heuristic Cropping method significantly outperforms both the Border Padding strategy and the single full image baseline.

nates. As presented in Table 5, our clustering-based method achieves a SS-Pro Avg. of 61.7, substantially outperforming both alternatives and the baseline. This result confirms that identifying spatial consensus through clustering is a more effective aggregation strategy.

Influence of View Resizing This experiment evaluates the impact of resizing cropped views to a higher resolution (2 $\times$ of the original size) to enhance the visibility of small target UI elements, thereby mitigating model instability on small regions. We compare the performance with and without this resizing operation. As shown in Table 6, resizing improves the SS-Pro Avg. score by 2.6 points (from 59.1 to 61.7), confirming its contribution to final performance.

Influence of View Number We compare the performance under different view numbers. As shown in Figure 4, we ob-

Aggregation Method	View Number	SS-Pro Avg.
Baseline (Single Full Image)	–	49.8
Average of Coordinates	4	46.6
Random Selection	4	55.7
Multi-Coordinate Clustering	4	61.7

Table 5. Comparison of aggregation strategies on the ScreenSpot-Pro benchmark using GTA1-7B. Multi-Coordinates Clustering method demonstrates superior performance, highlighting the importance of robust spatial consensus.

Method	Number of Views	SS-Pro Avg.
Baseline (Single Full Image)	–	49.8
Without Resizing	4	59.1
With Resizing	4	61.7

Table 6. Ablation study on view resizing using the ScreenSpot-Pro benchmark and GTA1-7B. Enlarging the cropped views to twice their original size improves the visibility of target UI elements and yields a performance gain of 2.6 points.

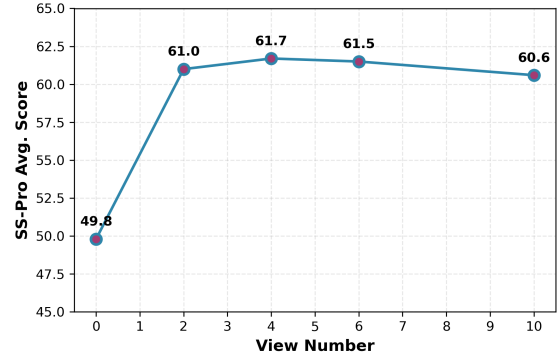


Figure 4. We evaluate GTA1-7B on ScreenSpot-Pro under different view number. Increasing the number of views does not consistently lead to performance improvements.

serve that increasing the number of views does not consistently lead to performance improvements. Although different views generate distinct coordinates, they tend to cluster around several fixed positions. Consequently, adding more views has limited impact on the final clustering results while increases the time cost.

6. Conclusion

We find that current existing grounding models are highly sensitive to visual input variations, leading to significant prediction instability and undermining model performance. To address this, we propose the training-free Multiple View Prediction (MVP) framework. MVP stabilizes predictions by cropping multiple views of screenshots, obtaining independent predictions, and aggregating them via spatial clustering. Extensive experiments on ScreenSpot-Pro, UI-Vision, and OSWorld-G validate that MVP consistently boosts the performance of diverse grounding models, effectively unleashing their potential without any retraining.

7. Acknowledgement

This work was partially supported by the National Natural Science Foundation of China (62441617) and was also supported in part by "Pioneer" and "Leading Goose" R&D Program of Zhejiang (No. 2024C01142). This work was supported by Ant Group and the Earth System Big Data Platform of the School of Earth Sciences, Zhejiang University.

References

- [1] anthropic. Introducing computer use, a new claude 3.5 sonnet, and claude 3.5 haiku, 2024. Accessed: 2024-10-22. [6](#)
- [2] Anthropic. Claude 3.7 sonnet and claude code, 2025. Accessed: 2025-02-25. [8](#)
- [3] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibao Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, Humen Zhong, Yuanzhi Zhu, Mingkun Yang, Zhao-hai Li, Jianqiang Wan, Pengfei Wang, Wei Ding, Zheren Fu, Yiheng Xu, Jiabo Ye, Xi Zhang, Tianbao Xie, Zesen Cheng, Hang Zhang, Zhibo Yang, Haiyang Xu, and Junyang Lin. Qwen2.5-vl technical report, 2025. [6](#), [7](#), [8](#)
- [4] Jikai Chen, Long Chen, Dong Wang, Leilei Gan, Chenyi Zhuang, and Jinjie Gu. V2p: From background suppression to center peaking for robust gui grounding task, 2025. [3](#), [6](#)
- [5] Liangyu Chen, Hanzhang Zhou, Chenglin Cai, Jianan Zhang, Panrong Tong, Quyu Kong, Xu Zhang, Chen Liu, Yuqi Liu, Wenxuan Wang, Yue Wang, Qin Jin, and Steven Hoi. Ui-ins: Enhancing gui grounding with multi-perspective instruction-as-reasoning, 2025. [2](#), [6](#)
- [6] Kanzhi Cheng, Qiushi Sun, Yougang Chu, Fangzhi Xu, Li YanTao, Jianbing Zhang, and Zhiyong Wu. SeeClick: Harnessing GUI grounding for advanced visual GUI agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9313–9332, Bangkok, Thailand, 2024. Association for Computational Linguistics. [1](#), [2](#), [6](#)
- [7] Boyu Gou, Ruohan Wang, Boyuan Zheng, Yanan Xie, Cheng Chang, Yiheng Shu, Huan Sun, and Yu Su. Navigating the digital world as humans do: Universal visual grounding for GUI agents. In *The Thirteenth International Conference on Learning Representations*, 2025. [2](#), [6](#), [7](#), [8](#)
- [8] Zhangxuan Gu, Zhengwen Zeng, Zhenyu Xu, Xingran Zhou, Shuheng Shen, Yunfei Liu, Beitong Zhou, Changhua Meng, Tianyu Xia, Weizhi Chen, Yue Wen, Jingya Dou, Fei Tang, Jinzhen Lin, Yulin Liu, Zhenlin Guo, Yichen Gong, Heng Jia, Changlong Gao, Yuan Guo, Yong Deng, Zhenyu Guo, Liang Chen, and Weiqiang Wang. Ui-venus technical report: Building high-performance ui agents with rft, 2025. [1](#), [6](#), [7](#), [8](#)
- [9] Dong Guo, Faming Wu, and etc. Feida Zhu. Seed1.5-vl technical report, 2025. [6](#), [7](#)
- [10] Wenyi Hong, Weihang Wang, Qingsong Lv, Jiazheng Xu, Wenmeng Yu, Junhui Ji, Yan Wang, Zihan Wang, Yuxuan Zhang, Juanzi Li, Bin Xu, Yuxiao Dong, Ming Ding, and Jie Tang. Cogagent: A visual language model for gui agents. In *Conference on Computer Vision and Pattern Recognition*, 2024. [1](#)
- [11] Xiaohu Huang, Hao Zhou, and Kai Han. Prunevid: Visual token pruning for efficient video large language models. In *Annual Meeting of the Association for Computational Linguistics*, 2025. [4](#)
- [12] Zheng Hui, Yinheng Li, Dan Zhao, Colby Banbury, Tianyi Chen, and Kazuhito Koishida. WinSpot: GUI grounding benchmark with multimodal large language models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 1086–1096, Vienna, Austria, 2025. Association for Computational Linguistics. [1](#)
- [13] Bin Lei, Nuo Xu, Ali Payani, Mingyi Hong, Chunhua Liao, Yu Cao, and Caiwen Ding. GUI-SPOTLIGHT: Adaptive iterative focus refinement for enhanced gui visual grounding, 2025. [2](#), [6](#), [7](#), [8](#)
- [14] Kaixin Li, Meng Ziyang, Hongzhan Lin, Ziyang Luo, Yuchen Tian, Jing Ma, Zhiyong Huang, and Tat-Seng Chua. Screenspot-pro: GUI grounding for professional high-resolution computer use. In *Workshop on Reasoning and Planning for Large Language Models*, 2025. [1](#), [2](#), [3](#), [6](#)
- [15] Zhecheng Li, Guoxian Song, Yiwei Wang, Zhen Xiong, Jun-song Yuan, and Yujun Cai. Generalist scanner meets specialist locator: A synergistic coarse-to-fine framework for robust gui grounding, 2025. [6](#)
- [16] Kevin Qinghong Lin, Linjie Li, Difei Gao, Zhengyuan Yang, Shiwei Wu, Zechen Bai, Weixian Lei, Lijuan Wang, and Mike Zheng Shou. Showui: One vision-language-action model for gui visual agent. In *Conference on Computer Vision and Pattern Recognition*, 2025. [2](#)
- [17] Junpeng Liu, Yifan Song, Bill Yuchen Lin, Wai Lam, Graham Neubig, Yuanzhi Li, and Xiang Yue. Visualwebbench: How far have multimodal llms evolved in web page understanding and grounding?, 2024. [1](#)
- [18] Run Luo, Lu Wang, Wanwei He, Longze Chen, Jiaming Li, and Xiaobo Xia. Gui-r1 : A generalist r1-style vision-language action model for gui agents, 2025. [2](#)
- [19] Tiange Luo, Lajanugen Logeswaran, Justin Johnson, and Honglak Lee. Visual test-time scaling for gui agent grounding, 2025. [2](#), [6](#)
- [20] Shravan Nayak, Xiangru Jian, Kevin Qinghong Lin, Juan A. Rodriguez, Montek Kalsi, Rabiul Awal, Nicolas Chapados, M. Tamer Özsu, Aishwarya Agrawal, David Vazquez, Christopher Pal, Perouz Taslakian, Spandana Gella, and Sai Rajeswar. Ui-vision: A desktop-centric gui benchmark for visual perception and interaction. In *Conference on International Conference on Machine Learning*, 2025. [2](#), [6](#), [8](#)
- [21] Dang Nguyen, Jian Chen, Yu Wang, Gang Wu, Namyong Park, Zhengmian Hu, Hanjia Lyu, Junda Wu, Ryan Aponte, Yu Xia, Xintong Li, Jing Shi, Hongjie Chen, Viet Dac Lai, Zhouhang Xie, Sungchul Kim, Ruiyi Zhang, Tong Yu, Mehrab Tanjim, Nesreen K. Ahmed, Puneet Mathur, Seunghyun Yoon, Lina Yao, Branislav Kveton, Jihyung Kil, Thien Huu Nguyen, Trung Bui, Tianyi Zhou, Ryan A. Rossi, and Franck Dernoncourt. Gui agents: A survey. In *Proceedings of Annual Meeting of the Association for Computational Linguistics*, 2025. [1](#)
- [22] OpenAI. Hello GPT-4o, 2024. Accessed: 2024-05-20. [6](#), [8](#)

- [23] OpenAI. Introducing operator, 2025. Accessed: 2025-01-23. 7
- [24] Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, Shizuo Tian, Junda Zhang, Jiahao Li, Yunxin Li, Shijue Huang, Wanjun Zhong, Kuanye Li, Jiale Yang, Yu Miao, Woyu Lin, Longxiang Liu, Xu Jiang, Qianli Ma, Jingyu Li, Xiaojun Xiao, Kai Cai, Chuang Li, Yaowei Zheng, Chaolin Jin, Chen Li, Xiao Zhou, Minchao Wang, Haoli Chen, Zhaojian Li, Haihua Yang, Haifeng Liu, Feng Lin, Tao Peng, Xin Liu, and Guang Shi. Ui-tars: Pioneering automated gui interaction with native agents, 2025. 1, 2, 7, 8
- [25] qwen. Qwen3vl, 2025. 2, 6
- [26] ByteDance Seed. Ui-tars-1.5. <https://seed-tars.com/1.5>, 2025. 2, 6
- [27] Fei Tang, Zhangxuan Gu, Zhengxi Lu, Xuyang Liu, Shuheng Shen, Changhua Meng, Wen Wang, Wenqi Zhang, Yongliang Shen, Weiming Lu, Jun Xiao, and Yueting Zhuang. Gui-g²: Gaussian reward modeling for gui grounding, 2025. 2
- [28] Fei Tang, Haolei Xu, Hang Zhang, Siqi Chen, Xingyu Wu, Yongliang Shen, Wenqi Zhang, Guiyang Hou, Zeqi Tan, Yuchen Yan, Kaitao Song, Jian Shao, Weiming Lu, Jun Xiao, and Yueting Zhuang. A survey on (m)llm-based gui agents, 2025. 1
- [29] Gemini Team, Rohan Anil, Sebastian Borgeaud, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M. Dai, Anja Hauth, and et al. Gemini: A family of highly capable multimodal models, 2024. 7
- [30] Venus Team, Changlong Gao, Zhangxuan Gu, Yulin Liu, Xinyu Qiu, Shuheng Shen, Yue Wen, Tianyu Xia, Zhenyu Xu, Zhengwen Zeng, Beitong Zhou, Xingran Zhou, Weizhi Chen, Sunhao Dai, Jingya Dou, Yichen Gong, Yuan Guo, Zhenlin Guo, Feng Li, Qian Li, Jinzhen Lin, Yuqi Zhou, Linchao Zhu, Liang Chen, Zhenyu Guo, Changhua Meng, and Weiqiang Wang. Ui-venus-1.5 technical report, 2026. 1
- [31] Shuai Wang, Weiwen Liu, Jingxuan Chen, Yuqi Zhou, Weinan Gan, Xingshan Zeng, Yuhang Che, Shuai Yu, Xinlong Hao, Kun Shao, Bin Wang, Chuhan Wu, Yasheng Wang, Ruiming Tang, and Jianye Hao. Gui agents with foundation models: A comprehensive survey, 2025. 1
- [32] Suyuchen Wang, Tianyu Zhang, Ahmed Masry, Christopher Pal, Spandana Gella, Bang Liu, and Perouz Taslakian. Improving gui grounding with explicit position-to-coordinate mapping, 2025. 1
- [33] Xuehui Wang, Zhenyu Wu, JingJing Xie, Zichen Ding, Bowen Yang, Zehao Li, Zhaoyang Liu, Qingyun Li, Xuan Dong, Zhe Chen, Weiyun Wang, Xiangyu Zhao, Jixuan Chen, Haodong Duan, Tianbao Xie, Chenyu Yang, Shiqian Su, Yue Yu, Yuan Huang, Yiqian Liu, Xiao Zhang, Yanting Zhang, Xiangyu Yue, Weijie Su, Xizhou Zhu, Wei Shen, Jifeng Dai, and Wenhao Wang. Mmbench-gui: Hierarchical multi-platform evaluation framework for gui agents, 2025. 1
- [34] Qianhui Wu, Kanzhi Cheng, Rui Yang, Chaoyun Zhang, Jianwei Yang, Huiqiang Jiang, Jian Mu, Baolin Peng, Bo Qiao, Reuben Tan, Si Qin, Lars Liden, Qingwei Lin, Huan Zhang, Tong Zhang, Jianbing Zhang, Dongmei Zhang, and Jianfeng Gao. Gui-actor: Coordinate-free visual grounding for gui agents, 2025. 1, 3, 6
- [35] Zhiyong Wu, Zhenyu Wu, Fangzhi Xu, Yian Wang, Qiushi Sun, Chengyou Jia, Kanzhi Cheng, Zichen Ding, Liheng Chen, Paul Pu Liang, and Yu Qiao. Os-atlas: A foundation action model for generalist gui agents, 2024. 2, 7, 8
- [36] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. Oworld: Benchmarking multimodal agents for open-ended tasks in real computer environments, 2024. 6
- [37] Tianbao Xie, Jiaqi Deng, Xiaochuan Li, Junlin Yang, Haoyuan Wu, Jixuan Chen, Wenjing Hu, Xinyuan Wang, Yuhui Xu, Zekun Wang, Yiheng Xu, Junli Wang, Doyen Sahoo, Tao Yu, and Caiming Xiong. Scaling computer-use grounding via user interface decomposition and synthesis. In *Conference on Neural Information Processing Systems*, 2025. 2, 6, 7
- [38] Long Xing, Qidong Huang, Xiaoyi Dong, Jiajie Lu, Pan Zhang, Yuhang Zang, Yuhang Cao, Conghui He, Jiaqi Wang, Feng Wu, and Dahua Lin. Pyramidrop: Accelerating your large vision-language models via pyramid visual redundancy reduction. In *Conference on Computer Vision and Pattern Recognition*, 2025. 4
- [39] Hai-Ming Xu, Qi Chen, Lei Wang, and Lingqiao Liu. Attention-driven gui grounding: Leveraging pretrained multimodal large language models without fine-tuning. In *The 39th Annual AAAI Conference on Artificial Intelligence*, 2025. 3, 4
- [40] Yiheng Xu, Zekun Wang, Junli Wang, Dunjie Lu, Tianbao Xie, Amrita Saha, Doyen Sahoo, Tao Yu, and Caiming Xiong. Aguis: Unified pure vision agents for autonomous gui interaction. In *International Conference on Machine Learning*, 2025. 7
- [41] Yan Yang, Dongxu Li, Yutong Dai, Yuhao Yang, Ziyang Luo, Zirui Zhao, Zhiyuan Hu, Junzhe Huang, Amrita Saha, Zeyuan Chen, Ran Xu, Liyuan Pan, Silvio Savarese, Caiming Xiong, and Junnan Li. Gta1: Gui test-time scaling agent, 2025. 2, 3, 6, 7
- [42] Yuhao Yang, Yue Wang, Dongxu Li, Ziyang Luo, Bei Chen, Chao Huang, and Junnan Li. Aria-ui: Visual grounding for gui instructions. In *Proceedings of Annual Meeting of the Association for Computational Linguistics*, 2025. 2
- [43] Xianhang Ye, Yiqing Li, Wei Dai, Miancan Liu, Ziyuan Chen, Zhangye Han, Hongbo Min, Jinkui Ren, Xiantao Zhang, Wen Yang, and Zhi Jin. Gui-arp: Enhancing grounding with adaptive region perception for gui agents, 2025. 2
- [44] Xinbin Yuan, Jian Zhang, Kaixin Li, Zhuoxuan Cai, Lujian Yao, Jie Chen, Enguang Wang, Qibin Hou, Jinwei Chen, Peng-Tao Jiang, and Bo Li. Enhancing visual grounding for gui agents via self-evolutionary reinforcement learning. In *Conference on Neural Information Processing Systems*, 2025. 6
- [45] Chaoyun Zhang, Shilin He, Jiaxu Qian, Bowen Li, Liqun Li, Si Qin, Yu Kang, Minghua Ma, Guyue Liu, Qingwei Lin, Saravan Rajmohan, Dongmei Zhang, and Qi Zhang. Large language model-brained gui agents: A survey, 2025. 1
- [46] Miaosen Zhang, Ziqiang Xu, Jialiang Zhu, Qi Dai, Kai Qiu, Yifan Yang, Chong Luo, Tianyi Chen, Justin Wagle, Tim

- Franklin, and Baining Guo. Phi-ground tech report: Advancing perception in gui grounding, 2025. [8](#)
- [47] Shaojie Zhang, Pei Fu, Ruoceng Zhang, Jiahui Yang, Anan Du, Xiuwen Xi, Shaokang Wang, Ying Huang, Bin Qin, Zhenbo Luo, and Jian Luan. Hyperclick: Advancing reliable gui grounding via uncertainty calibration, 2025. [6](#), [8](#)
- [48] Yunzhu Zhang, Yu Lu, Tianyi Wang, Fengyun Rao, Yi Yang, and Linchao Zhu. Flexselect: Flexible token selection for efficient long video understanding. In *Conference on Neural Information Processing Systems*, 2025. [4](#), [6](#)
- [49] Yu Zhao, Wei-Ning Chen, Huseyin Atahan Inan, Samuel Kessler, Lu Wang, Lukas Wutschitz, Fangkai Yang, Chaoyun Zhang, Pasquale Minervini, Saravan Rajmohan, and Robert Sim. Learning gui grounding with spatial reasoning from visual feedback, 2025. [6](#)
- [50] Yu Zhao, Wei-Ning Chen, Huseyin Atahan Inan, Samuel Kessler, Lu Wang, Lukas Wutschitz, Fangkai Yang, Chaoyun Zhang, Pasquale Minervini, Saravan Rajmohan, and Robert Sim. Learning gui grounding with spatial reasoning from visual feedback, 2025. [1](#), [2](#)
- [51] Boyuan Zheng, Boyu Gou, Jihyung Kil, Huan Sun, and Yu Su. Gpt-4v(ision) is a generalist web agent, if grounded. In *Forty-first International Conference on Machine Learning*, 2024. [1](#)